

Apache Cassandra: teoria e pratica

Progetto di Architetture Dati

Coldani Andrea mat.894684, Colonetti Fabio mat.899689

18 Giugno 2026

[Link al codice del progetto](#)

Indice

1	Introduzione e nozioni base	3
1.1	Cap theorem	3
1.2	Bloom Filter	4
2	Modello dei Dati e Struttura Logica	5
2.1	Modellazione dei Dati Query-Driven	5
2.2	Meccanismi di Partizionamento e Scalabilità	6
2.2.1	La Chiave Primaria	6
2.2.2	Dal Modulo al Consistent Hashing	6
2.3	Disponibilità e Protocolli Peer-to-Peer	7
2.3.1	Protocollo Gossip	7
2.3.2	Allineamento repliche	7
2.3.3	Fattore di Replicazione	8
2.4	Architettura di Scrittura e Lettura	8
2.4.1	Processo di Scrittura	9
2.4.2	Percorso di Lettura	9
3	Sviluppo e Progettazione Database	10
3.1	Dominio applicativo	10
3.2	Architettura codice	11
3.3	schema.cql	11
3.4	docker-compose.yml	12
3.5	dbLogic.py	12
3.6	workload*.py	13
4	Performance e Risultati	15
4.1	Test Senza Variazione del Numero di Nodi	15
4.2	Test con Variazione del Numero di Nodi	16
4.3	Test su Macchine Distinte	17

5	Conclusioni	18
5.1	Analisi delle Performance e Carico dei Nodi	18
5.2	Scalabilità e Vincoli Hardware	19
5.3	Tolleranza ai Guasti e Consistenza	19
6	How to Run	20
6.1	Avvio del cluster Cassandra tramite Docker	20
6.2	Caricamento dello schema	20
6.3	Popolamento del database	21
6.4	Accesso alla shell CQL	21
6.5	Gestione dei container	21

1 Introduzione e nozioni base

Apache Cassandra nasce nel 2007 presso Facebook per opera di **Avinash Lakshman** (già co-creatore di Amazon Dynamo) con l'obiettivo di risolvere le limitazioni strutturali del sistema di messaggistica interna. La sfida consisteva nel gestire miliardi di messaggi per milioni di utenti con una latenza minima e una disponibilità totale.

Cassandra rappresenta una sintesi ingegneristica tra due modelli fondamentali: il design distribuito di **Amazon Dynamo** e il modello di archiviazione dei dati di **Google BigQuery**.

È basata su un'architettura distribuita multi nodo con la proprietà di **Transparent Elasticity**, infatti è possibile aggiungere nodi online senza impatti sul funzionamento degli altri nodi.

Cassandra è progettata per essere una soluzione NoSQL altamente scalabile che elimina ogni *single point of failure* attraverso un'architettura decentralizzata.

1.1 Cap theorem

Il **CAP theorem** è molto significativo per comprendere gli aspetti fondamentali relativi a Cassandra.

Innanzitutto vanno definiti i seguenti concetti:

- **Consistency**: garantisce che tutti i nodi del sistema osservino una vista coerente dei dati. In particolare, ogni operazione di lettura restituisce l'ultimo valore scritto, indipendentemente dal nodo interrogato.
- **Availability**: assicura che ogni richiesta inviata a un nodo non guasto riceva sempre una risposta in un tempo finito. La risposta può non riflettere necessariamente lo stato più aggiornato dei dati, ma il sistema rimane operativo e reattivo.
- **Partition Tolerance**: indica la capacità del sistema di continuare a funzionare anche in presenza di guasti di rete che impediscono la comunicazione tra gruppi di nodi, generando partizioni isolate della rete.

CAP Theorem

Il **CAP theorem** stabilisce che un sistema distribuito non può fornire contemporaneamente strong **Consistency** (C), **Availability** (A) e **Partition Tolerance** (P) della rete.

Il CAP theorem può essere rappresentato mediante un **triangolo** i cui **vertici** corrispondono alle tre proprietà fondamentali Consistency (C), Availability (A) e Partition Tolerance (P), mentre i lati del triangolo rappresentano le possibili combinazioni tra due di esse. Il teorema stabilisce che un sistema distribuito non può posizionarsi contemporaneamente su tutti e tre i vertici del triangolo, ma deve necessariamente collocarsi su un solo lato.

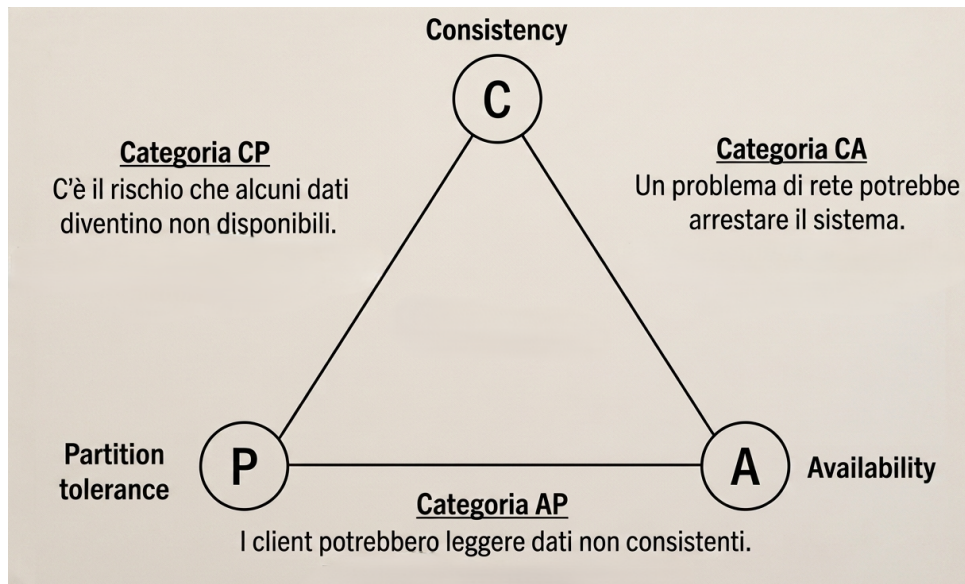


Figura 1: Cap theorem

Cassandra viene generalmente classificato come un sistema **AP** (Availability + Partition Tolerance), infatti privilegia la disponibilità del servizio e la tolleranza alle partizioni, accettando la possibilità di leggere dati non perfettamente aggiornati in alcune condizioni.

Tuttavia, grazie al modello di **consistenza configurabile** (Tunable Consistency), è possibile modulare il grado di coerenza dei dati definendo opportunamente le politiche di lettura e scrittura. In questo modo il sistema può avvicinarsi a un comportamento di tipo **CP** (Consistency + Partition Tolerance), a scapito della disponibilità in presenza di condizioni avverse della rete. In particolare le politiche di lettura e scrittura principali possono essere:

- **ONE:** Basta la conferma di una replica. Massima velocità, ma rischio di leggere dati vecchi.
- **QUORUM:** Richiede la conferma della maggioranza (es. 3 su 5). È il bilanciamento ideale per la maggior parte dei sistemi.
- **ALL:** Richiede la conferma di tutte le repliche. Garantisce consistenza forte, ma riduce la disponibilità (se un nodo è offline, la query fallisce).

1.2 Bloom Filter

Un **Bloom filter** è una struttura dati probabilistica utilizzata per verificare l'appartenenza di un elemento a un insieme. È progettato per essere estremamente efficiente in termini di memoria e tempo di risposta, al costo di una possibile presenza di **falsi positivi**, mentre **non produce mai falsi negativi**.

La struttura si basa su un array di m bit inizialmente impostati a 0 e su k funzioni di hash indipendenti. Ogni elemento inserito viene elaborato dalle k funzioni di hash, che producono k indici nell'array; i bit corrispondenti a tali posizioni vengono impostati a 1.

La fase di **query** avviene in modo analogo: l'elemento viene hashato con le stesse k funzioni e si controllano i bit nelle posizioni ottenute. Se **almeno uno** di questi bit è 0, l'elemento è **certamente non presente**. Se invece tutti i bit risultano 1, l'elemento viene considerato **potenzialmente presente**, con una certa probabilità di falso positivo. La probabilità di falso positivo dipende principalmente dalla dimensione dell'array m , dal numero di elementi inseriti n e dal numero di funzioni di hash k : in particolare, all'aumentare di n , cresce la saturazione dell'array di bit, aumentando la probabilità di collisioni tra elementi diversi.

2 Modello dei Dati e Struttura Logica

A differenza dei database relazionali, Cassandra adotta un approccio **Wide Column Store**. La gerarchia dei dati è organizzata come segue (un esempio in figura 2):

- **Keyspace:** Il contenitore di massimo livello (equivalente a un database in SQL). Definisce le impostazioni di replica e il numero di copie dei dati da mantenere nel cluster (si parlerà più approfonditamente di questo nella sezione 2.3.3).
- **Tabella (o Column Family):** Definisce lo schema, ovvero colonne e tipi, ma è flessibile e quindi non richiede che ogni riga possieda le medesime colonne.
- **Righe e Colonne:** Ogni riga è identificata da una chiave primaria. Le colonne sono composte da un nome, un tipo e un valore. La flessibilità permette a righe diverse della stessa tabella di avere set di attributi differenti (es. una riga con indirizzo e una senza) senza necessità di valori nulli.

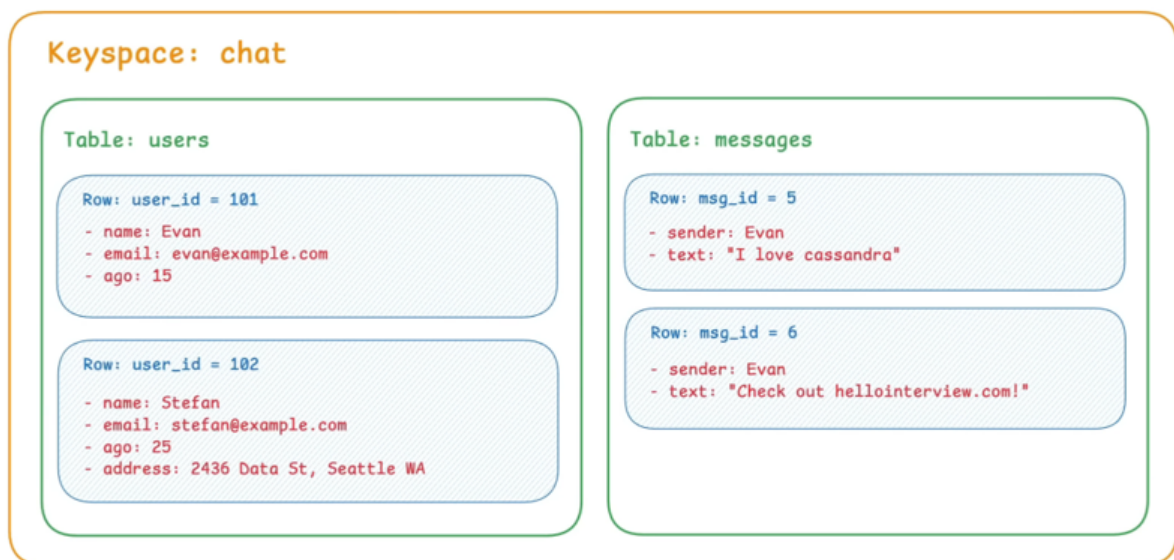


Figura 2: Gerarchia dei componenti

2.1 Modellazione dei Dati Query-Driven

In Cassandra, la modellazione dei dati segue un approccio **Query-Driven**, ovvero conviene progettare il database a partire dalle query che l'applicazione dovrà eseguire più

frequentemente, anziché dalla rappresentazione delle entità e delle loro relazioni come avviene nei database relazionali.

Per questo motivo, le tabelle vengono create e organizzate in modo da consentire l'accesso diretto ai dati necessari per uno specifico caso d'uso. Ciò porta spesso a una **denormalizzazione** dei dati, ovvero alla duplicazione controllata delle informazioni in più tabelle per ottimizzare le operazioni di lettura.

Questo approccio riduce la necessità di operazioni costose come **join** e aggregazioni, che Cassandra non supporta, garantendo prestazioni elevate e tempi di risposta prevedibili anche su grandi quantità di dati.

Cassandra risulta quindi particolarmente efficiente in sistemi con un **insieme limitato** e ben definito di **query**, dove i pattern di accesso ai dati sono noti a priori e i requisiti di scalabilità e disponibilità sono elevati.

2.2 Meccanismi di Partizionamento e Scalabilità

La scalabilità orizzontale di Cassandra é ottenuta grazie alla distribuzione dei dati tra i vari nodi, in modo che ognuno gestisca solo una frazione dell'intero dataset, collaborando con gli altri per aumentare la **availability**.

2.2.1 La Chiave Primaria

La chiave primaria è l'elemento cruciale e si divide in:

1. **Partition Key (Chiave di Partizione):** Determina la posizione fisica dei dati. Tutte le righe con la stessa chiave di partizione risiedono sullo stesso nodo per ottimizzare l'accesso. Ad esempio, usando **user_id** come chiave di partizione, tutti i messaggi di un utente specifico saranno raggruppati sullo stesso server.
2. **Clustering Key (Chiave di Clustering):** Determina l'ordinamento logico dei dati all'interno della partizione (es. ordinare i messaggi per timestamp).

2.2.2 Dal Modulo al Consistent Hashing

Inizialmente, i sistemi distribuiti usavano l'hash della chiave modulo il numero di nodi ($hash \pmod N$). Tuttavia, questo approccio causa uno shuffling massiccio di dati ogni volta che si aggiunge o rimuove un nodo. Cassandra risolve il problema con il **Consistent Hashing**:

- I valori di hash sono disposti su un **anello virtuale**.
- Ogni nodo è responsabile di un intervallo di questo anello.
- Quando una chiave viene inserita, il sistema calcola l'hash e procede in senso orario sull'anello fino a trovare il nodo giusto.

L'uso di **nodi virtuali (vnodes)** permette una distribuzione del carico ancora più uniforme.

Ogni dato è replicato nel nodo relativo all'intervallo calcolato tramite l'hashing ed a **RF** nodi successivi con **r replication factor** (si veda sezione 2.3.3).

Di seguito una rappresentazione grafica di quanto appena detto:

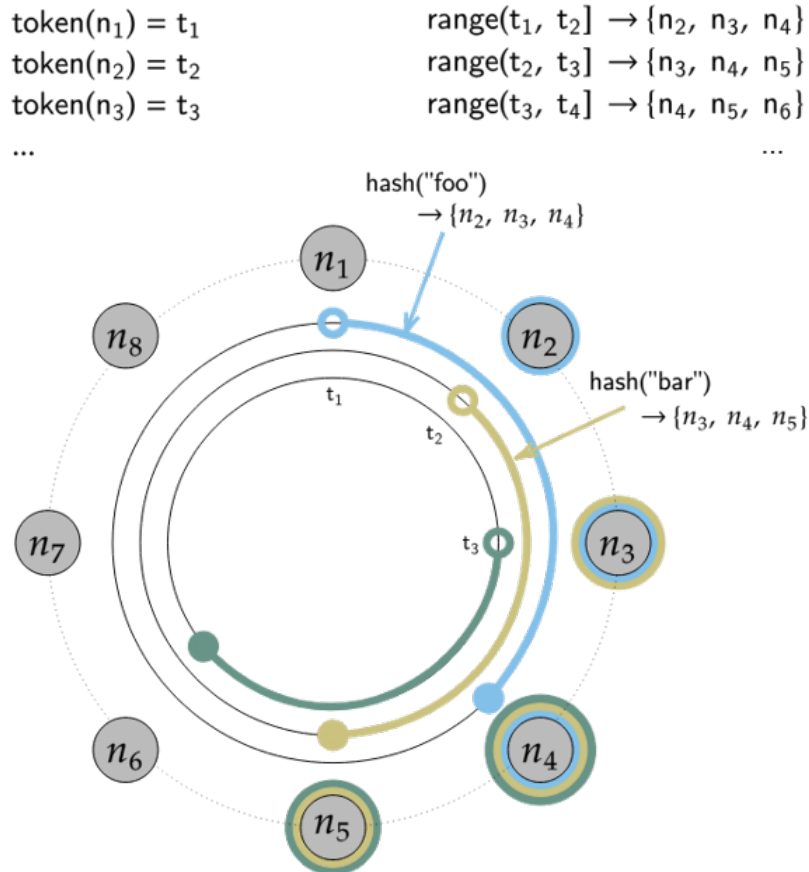


Figura 3: Struttura ad anello dei nodi Cassandra

2.3 Disponibilità e Protocolli Peer-to-Peer

Cassandra è un sistema **leaderless**, ovvero senza master: ogni replica è uguale alle altre e ogni nodo può coordinare una richiesta. Per tale ragione sono importanti gli elementi esposti nelle seguenti sezioni (sezioni 2.3.1 e 2.3.3).

2.3.1 Protocollo Gossip

Per mantenere il cluster sincronizzato senza un coordinatore centrale i nodi comunicano tramite il **Gossip Protocol**. A intervalli regolari, ogni nodo scambia piccole informazioni con alcuni nodi scelti casualmente riguardo lo stato del cluster (nodi attivi/inattivi, intervalli di dati posseduti, versioni dello schema): questo permette al cluster di auto-gestirsi e di evitare colli di bottiglia o punti di guasto centralizzati.

Questo meccanismo consente anche la **failure detection**, in particolare rileva l'assenza di comunicazione con altri nodi e li marca prima come *suspect* e poi come *down* se il problema persiste.

2.3.2 Allineamento repliche

L'allineamento delle repliche avviene tramite un meccanismo di **anti-entropy**, basato sull'utilizzo delle **Merkle Trees**. Tali strutture vengono impiegate durante il processo

di **repair** per confrontare in modo efficiente i dati tra repliche e rilevare eventuali inconsistenze.

Il loro funzionamento si basa sulla costruzione di un albero di hash: i dati vengono suddivisi in intervalli e, per ciascun intervallo, viene calcolato un valore di hash. Gli hash dei nodi foglia vengono poi combinati gerarchicamente fino a ottenere un hash radice. Durante il repair, i nodi confrontano questi hash; se il valore della radice coincide, le repliche sono coerenti, altrimenti il confronto viene approfondito ricorsivamente fino a individuare le differenze.

Di seguito in figura 4 un esempio di quanto detto:

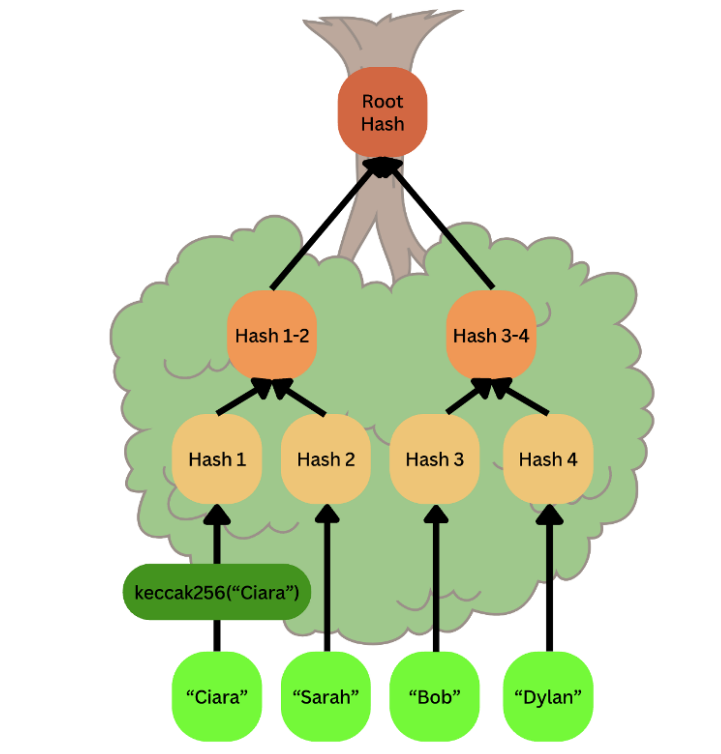


Figura 4: Esempio di Merkle-Tree

2.3.3 Fattore di Replicazione

Il **Replication Factor (RF)** determina quante copie esistono per ogni dato. Cassandra posiziona le repliche in modo strategico su diversi rack o data center per garantire che, anche in caso di blackout di un intero settore, i dati rimangano accessibili.

2.4 Architettura di Scrittura e Lettura

L'architettura descritta segue il modello **LSM-Tree** (Log-Structured Merge Tree), su cui si basa Cassandra: le scritture vengono prima registrate nel **Commit Log** e nella **Mem-Table** in memoria, per poi essere trasferite su disco in file immutabili chiamati **SSTable** (Sorted String Tables).

Questo approccio elimina la necessità di aggiornare direttamente i dati su disco, consentendo un throughput di scrittura molto elevato; infatti le SSTable sfruttano l'**append-only** ovvero i dati non vengono sovrascritti, ma vengono appesi e successivamente consolidati tramite il processo di **compattazione**. Quest'ultimo mantiene la versione più

recente di ciascun dato in base al timestamp basandosi su oggetti come le **tombstone**: essi sono dei marcatori di cancellazione che non eliminano immediatamente il dato, ma lo segnalano come rimosso, permettendo la loro eliminazione definitiva solo in seguito durante la compaction.

2.4.1 Processo di Scrittura

Cassandra raggiunge velocità di scrittura elevatissime perché evita IO casuali e lock. Il processo è il seguente:

1. Il nodo coordinatore riceve la richiesta e la invia alle repliche.
2. La replica scrive immediatamente nel **Commit Log** su disco (per la durabilità).
3. Successivamente scrive nella **MemTable** in RAM (pattern **WAL** write-after-log).
4. Una volta saturata la MemTable, i dati vengono scaricati (flush) asincronamente su disco in file immutabili chiamati **SS Tables** (Sorted String Tables).
5. Poiché le SS Table sono immutabili, gli aggiornamenti non sovrascrivono i dati esistenti, ma creano nuove voci unificate poi tramite la **compattazione** (compaction).

Di seguito uno schema di quanto detto:

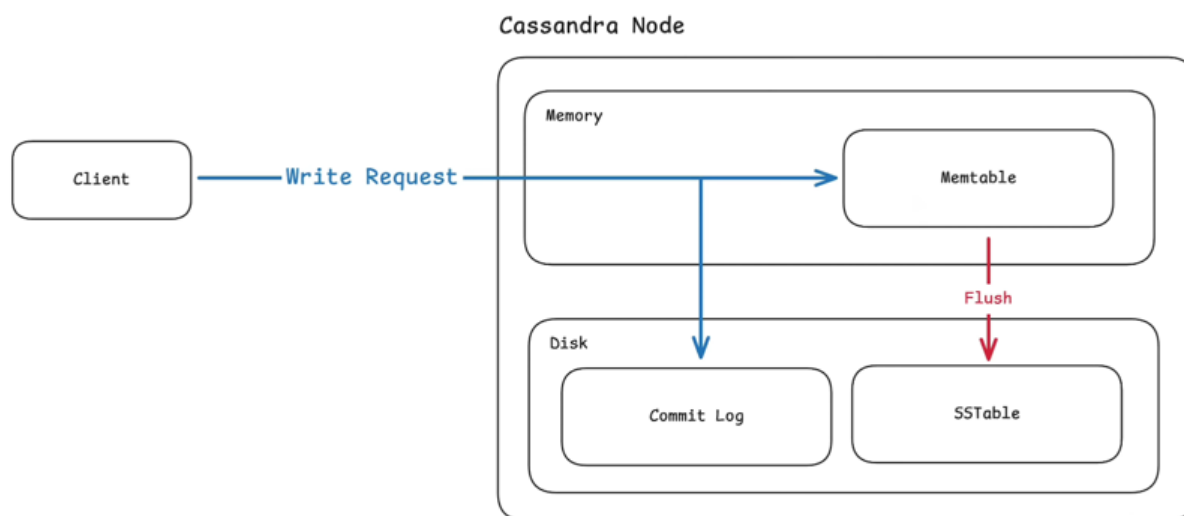


Figura 5: Schema di scrittura nel DB

2.4.2 Percorso di Lettura

La lettura è più complessa: il sistema deve consultare la MemTable e potenzialmente diverse SS Tables. Per mantenere le prestazioni:

- Utilizza **Bloom filter** per escludere file che sicuramente non contengono il dato e indici specifici.

I Bloom filter sono strutture dati **probabilistiche** che consentono di verificare

rapidamente se un elemento non è presente in un insieme: possono generare falsi positivi ma non falsi negativi. Per una descrizione più dettagliata vedere la sezione 1.2.

- Se le repliche restituiscono dati diversi, il coordinatore sceglie il valore con il timestamp più recente.
- Viene avviato un **Read Repair** in background per aggiornare le repliche che avevano dati obsoleti.

Di seguito uno schema di quanto detto:

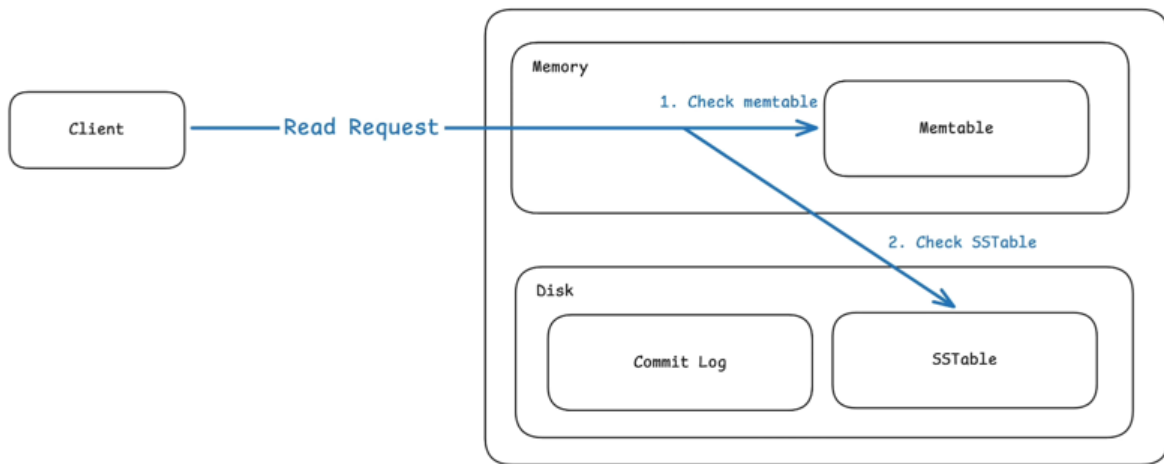


Figura 6: Schema di lettura nel DB

3 Sviluppo e Progettazione Database

3.1 Dominio applicativo

Il dominio riguarda la progettazione e gestione di un **social network** distribuito (Twitter-like), in cui gli utenti possono registrarsi, creare e condividere post anche con contenuti multimediali, seguire altri utenti e consultare un feed personalizzato dei contenuti pubblicati dalle persone che seguono.

I **contenuti multimediali** relativi ai post non vengono realmente memorizzati nel database Cassandra, ma sono rappresentati tramite generazioni di bit casuali o link fittizi, con lo scopo di simulare la presenza di foto e video che, in un'architettura reale, sarebbero tipicamente archiviati in sistemi esterni dedicati allo storage distribuito.

Un **utente** è definito da *username*, *email*, *password* hashata, *nome*, *cognome* e un'eventuale *immagine profilo*. In questo caso, dato che le immagini profilo sono tipicamente piccole, sono memorizzate come colonna della tabella, simulate come byte randomici. Un **post** è definito dallo *username* (ovvero l'utente che lo ha pubblicato), da un *id* (generato a partire dall'username e dal timestamp), dal *testo* e possibilmente una *risorsa multimediale*

e una *location*. Inoltre sono definite le tabelle **followers_by_user** e **following_by_user** per tracciare le relazioni tra utenti.

3.2 Architettura codice

Per analizzare il comportamento di Cassandra sono stati simulati n nodi, ciascuno eseguito all'interno di un diverso **Container Docker**. Questo ci consente un possibile incremento progressivo del numero di nodi, con l'aspettativa che le prestazioni del sistema crescano in modo approssimativamente lineare.

Successivamente sono state eseguite anche prove di **fault tolerance**, simulando la caduta di uno o più nodi al fine di verificare la robustezza del sistema e la sua capacità di continuare a operare correttamente.

Le componenti principali del codice sono:

- **dbLogic.py**: Gestisce la logica del database e le operazioni correlate per il suo setup;
- **docker-compose.yml**: avvia un cluster Cassandra a n nodi in Docker, configurando un cluster chiamato "CassandraCluster" con replica tra nodi, persistenza dei dati su volumi separati e controlli di healthcheck e dipendenze di avvio tra i servizi;
- **schema.cql**: definisce il keyspace e le tabelle del database, modellando utenti, post, feed e relazioni di follower/following ottimizzati per query specifiche: profilo, timeline e rete sociale;
- **workload*.py**: si occupa del popolamento dei dati e della simulazione di operazioni su diverse configurazioni del sistema, al fine di riprodurre scenari realistici di utilizzo.

Nelle sezioni seguenti saranno analizzati più approfonditamente alcuni componenti di interesse.

3.3 schema.cql

Definisce un keyspace Cassandra denominato "**network_giustino**", al suo interno si può configurare la strategia di replica ed il fattore di replica (visto nella sezione 2.3.3).

All'interno del keyspace vengono definite diverse tabelle per supportare le funzionalità principali di un social network. La tabella **users** memorizza le informazioni degli utenti, identificati univocamente tramite **username**, includendo un'immagine profilo salvata come blob e informazioni anagrafiche come nome, cognome, l'indirizzo email, la password sotto forma di hash implementato con SHA256.

La tabella **posts_by_user** modella i post degli utenti, utilizzando una chiave di partizione basata su **username** e una chiave di clustering **post_id** di tipo **timeuuid** ordinata in modo decrescente per consentire il recupero rapido dei post più recenti (si veda il significato delle chiavi alla sezione 2.2.1).

La tabella `home.feed` è progettata per ottimizzare la lettura del feed personale, precomputando i contenuti visibili a ciascun utente e ordinandoli per data.

Le relazioni sociali sono gestite tramite due tabelle denormalizzate: `following_by_user`, che memorizza gli utenti seguiti da ciascun utente, e `followers_by_user`, che rappresenta invece gli utenti che seguono un determinato profilo.

Questa struttura segue il paradigma di Cassandra basato sulla denormalizzazione e sulla progettazione orientata alle query (si veda sezione 2.1 per più dettagli).

3.4 `docker-compose.yml`

È un file di configurazione e definisce i nodi del cluster Cassandra con l'obiettivo di simulare il sistema in locale.

Tutti i nodi utilizzano l'immagine ufficiale `cassandra:4.1` e appartengono allo stesso cluster logico denominato `CassandraCluster`, garantendo così la possibilità di replicare dati e testare scenari di consistenza e tolleranza ai guasti.

Il primo nodo funge da *seed node*, ovvero punto di riferimento iniziale per la formazione del cluster, mentre gli altri nodi si collegano ad esso tramite la variabile `CASSANDRA_SEEDS`. La configurazione utilizza lo `SimpleSnitch`, adatta a deployment single-datacenter, coerente con un ambiente di sviluppo.

Un aspetto rilevante è la gestione delle risorse JVM, dove vengono impostati esplicitamente i limiti di heap (`MAX_HEAP_SIZE` e `HEAP_NEWSIZE`) per ottimizzare le prestazioni in un contesto containerizzato. Inoltre, sul primo nodo viene abilitato l'accesso JMX remoto tramite `LOCAL_JMX=no` e parametri JVM aggiuntivi, permettendo il monitoraggio del cluster.

Per garantire una corretta inizializzazione, i nodi sono collegati tramite `depends_on` e `healthcheck`, che verificano la disponibilità del servizio Cassandra tramite comandi CQL prima di avviare i nodi successivi.

Vengono inoltre esposte le porte 9042 e 7199: la prima è utilizzata da Cassandra per le connessioni client tramite protocollo CQL, mentre la seconda è dedicata al monitoraggio del sistema attraverso JMX, consentendo l'accesso a metriche e strumenti di analisi. Osservazione: tramite il comando `"jconsole"` e collegandosi a `"localhost:7199"` si otterrà una visualizzazione grafica delle performance

Infine, vengono definiti volumi persistenti separati per ciascun nodo, assicurando la durabilità dei dati anche in caso di riavvio dei container.

3.5 `dbLogic.py`

Implementa la logica applicativa: la classe principale `DBLogic` gestisce la connessione al cluster, la configurazione del keyspace e la preparazione delle query CQL, ottimizzando le prestazioni tramite l'uso di **prepared statements**, ovvero query che vengono precompilate e analizzate una sola volta dal database. In fase di esecuzione, tali query ricevono esclusivamente i parametri dinamici, evitando così la ripetizione delle operazioni

di parsing e pianificazione dell'esecuzione ad ogni richiesta e riducendo significativamente l'overhead computazionale.

Una parte centrale del sistema è la generazione di dati realistici, realizzata tramite la libreria **Faker**. Vengono creati utenti, post e relazioni sociali (following/followers) con **distribuzioni probabilistiche** che simulano comportamenti realistici, come il numero variabile di post per utente e la presenza opzionale di contenuti multimediali (per esempio un post deve avere un testo mentre una risorsa audio/video non é obbligatoria). Ad esempio, per quanto riguarda la creazione degli utenti, viene usata questa distribuzione: l'utente non ha una foto profilo il 50% delle volte, nei restanti casi, può presentare con la stessa probabilità una foto profilo che pesa rispettivamente 1Kb, 5KB, 10KB. Oppure, per quanto riguarda il numero di post per ciascun utente, la distribuzione segue una curva tipica dei social, in particolare una buona parte degli utenti ha 0/1 post pubblicati, pochi ne hanno alcuni e ancora meno hanno molti post. La stessa logica è stata seguita anche per il numero di seguiti.

La libreria **Faker** può generare potenziali duplicati, poiché attinge da un insieme limitato di dati predefiniti. Per questo motivo è stato necessario introdurre una struttura dati di tipo **set**, utilizzata per garantire l'unicità degli identificativi (in particolare degli username), evitando così inserimenti duplicati e mantenendo l'integrità delle chiavi primarie nel database.

Per migliorare le prestazioni in fase di popolamento del database, il codice utilizza **esecuzioni concorrenti** tramite il metodo del driver Python di Cassandra `execute_concurrent_with_args`, riducendo significativamente il tempo necessario per inserire grandi quantità di dati: infatti, per inserimenti con più di 15 elementi (parametro opportunamente scelto), l'utilizzo del metodo consente di eseguire più operazioni in parallelo anziché in modo sequenziale, in questo modo si riduce il tempo complessivo di esecuzione e si migliora l'utilizzo delle risorse di rete e del client.

Inoltre, viene implementata una strategia di **fan-out on write** per la home feed, precomputando le timeline degli utenti in base ai loro follower (maggiori dettagli nella sezione 3.6).

La logica include anche la gestione delle relazioni sociali attraverso tabelle denormalizzate, coerenti con il modello di Cassandra, aggiornate simultaneamente per garantire coerenza tra viste complementari (following e followers).

Infine, sono presenti funzioni di inserimento singolo e batch per utenti, post e relazioni, che permettono sia test mirati sia generazione massiva di dati per benchmarking del sistema.

3.6 workload*.py

Implementa gli **scenari** di utilizzo volti a simulare il comportamento realistico degli utenti di un social network e a misurare le prestazioni del cluster Cassandra in termini di throughput e latenza in varie situazioni.

Generalmente per ogni scenario viene inizializzato il database con un insieme di utenti, post e relazioni sociali generate automaticamente. Solo in alcuni scenari il sistema avvia

più **thread**, per simulare anche la **concorrenza**.

Particolare attenzione è dedicata alla simulazione della strategia **fan-out on write**, come detto nella sezione 3.5. Quando un utente pubblica un nuovo contenuto, il sistema recupera dal database l'elenco reale dei suoi follower e inserisce immediatamente una copia del post nella home feed di ciascuno di essi. Questa scelta consente di valutare il costo computazionale della propagazione dei contenuti in fase di scrittura, approccio comunemente adottato nei social network per velocizzare le successive operazioni di lettura.

Al termine dell'esecuzione vengono raccolte e analizzate le **metriche** principali del sistema, tra cui il numero totale di operazioni completate, il throughput espresso in operazioni al secondo e la latenza media delle richieste. Tali misure consentono di valutare il comportamento del cluster Cassandra sotto un carico concorrente e di confrontare l'impatto delle diverse configurazioni o strategie di modellazione dei dati.

Questo componente gestisce le interazioni effettive con il database e le operazioni su di esso; non è quindi costituito da una singola classe, ma da un insieme di classi, come illustrato di seguito:

- **workloadBase**: inizializza il database con 1000 utenti e simula che 50 di questi effettuino ogni 0.5 secondi delle operazioni di lettura o scrittura. In particolare sono state effettuate le principali operazioni tipiche di un social network secondo la seguente **distribuzione probabilistica**: consultazione della propria home feed (50% delle operazioni), pubblicazione di un post (30%) oppure creazione di una nuova relazione di follow (20%). In questo modo il benchmark riproduce un carico misto di letture e scritture, più vicino a uno scenario reale rispetto a test sintetici basati su una singola tipologia di query;
- **workloadConsistency**: partendo da database inizializzato con 100 utenti, carica 5000 utenti e effettua 1000 operazioni casuali senza concorrenza testando le performance sui 3 diversi **consistency level**;
- **workloadImagePayload**: partendo da database vuoto, carica 1000 utenti 2 volte, la prima gli utenti hanno un immagine profilo che pesa 1KB, mentre la seconda 100KB. Questo scenario serve appositamente per testare le performance a differenza di payload degli utenti;
- **workloadReadMiss**: inizializza il database con 100 utenti e simula su utenti esistenti e non esistenti per testare le performance relative a **hit** e **miss** nel database;
- **workloadStressTest**: partendo dal database inizializzato con 1000 utenti, simula che 100 utenti (threads) effettuino 500 caricamenti ognuno di post in parallelo alla lettura del feed per testare una situazione di stress in lettura e scrittura;
- **workloadScalability**: è uno script shell che utilizza Docker per automatizzare la configurazione ed esecuzione di nodi distinti per poter testare il comportamento all'**aumentare dei nodi**. In particolare, avvia il nodo 1 ed esegue il *workloadStressTest*; successivamente avvia il nodo 2 e ripete lo stesso test. Infine, procede con l'avvio dei nodi 3 e 4, eseguendo nuovamente il *workloadStressTest*;

- **workloadFaultTolerance***: si simula il carico di un utente che esegue un'operazione al secondo, testando il sistema in diverse configurazioni del **consistency level** (si veda Sezione 1.1): pertanto questo workload é suddiviso in 3 file (*workloadFaultToleranceONE.py*, *workloadFaultToleranceQUORUM.py*, *workloadFaultToleranceALL.py*). Inoltre, per ognuno dei 3 consistency level è stata valutata la **caduta dei nodi**: per un cluster di n nodi, sono state eseguite 5 richieste, dopodiché è stato fatto fallire un nodo; successivamente sono state inviate altre 5 richieste e si è proceduto alla caduta di un ulteriore nodo. Tale processo è stato ripetuto per $n - 1$ iterazioni, fino al guasto progressivo di tutti i nodi tranne uno. Questa procedura è stata eseguita per ciascun consistency level considerato.
- **workloadMemStressTest**: questo workload vuole testare la quantità di memoria utilizzabile prima del degravamento delle performance (latenza), quindi carica 1000 utenti alla volta con 1KB di payload, finché la latenza non degrada di un fattore 2 per più di 3 inserimenti.

I risultati e le performance per ogni scenario implementato dai workload sono esposti nella sezione 4.

4 Performance e Risultati

I risultati sono relativi ai diversi **workload**, che rappresentano differenti scenari operativi potenzialmente riscontrabili in un social network e che sono stati riprodotti attraverso tali workload (si veda la sezione 3.6 per maggiori dettagli sul loro funzionamento individuale).

4.1 Test Senza Variazione del Numero di Nodi

Tutti i test non volti a valutare la variabilità del numero di nodi sono stati eseguiti con un numero di nodi $n = 4$ e *Replication Factor* = 3 come da configurazione standard (si veda sezione 2.2.2 per maggiori dettagli in merito), poiché rappresenta il massimo consentito dall'hardware a disposizione (in particolare, un PC con 16 GB di RAM, processore i5 e GPU GTX 1050 Ti).

Di seguito le **performance** per ogni singolo scenario:

Metric	Value
Actual Execution Time	60.52 s
Total Completed Actions	4343 ops
System Throughput	71.76 ops/s
Average Action Latency	6.84 ms

Tabella 1: workloadBase

Level	Avg Write Latency	Avg Read Latency	Ops (W/R)
ONE	1.71 ms	2.58 ms	3000/3000
QUORUM	1.83 ms	2.94 ms	2987/3000
ALL	1.92 ms	2.97 ms	2993/3000

Tabella 2: workloadConsistency

Performance Metric	1 KB PAYLOAD	100 KB PAYLOAD
Total Completed Insertions	1000 ops	1000 ops
Total Execution Time	2.06 s	11.05 s
System Throughput	486.03 ops/s	90.46 ops/s
Average Write Latency	2.06 ms	11.05 ms

Tabella 3: workloadImagePayload

Performance Metric	READ HIT	READ MISS
Total Completed Operations	1000 ops	1000 ops
Total Execution Time	2.20 s	2.32 s
System Throughput	453.70 ops/s	431.72 ops/s
Average Request Latency	2.20 ms	2.31 ms

Tabella 4: workloadReadMiss

Performance Metric	INTENSE WRITING	INTENSE READING
Total Completed Operations	50000 ops	50000 ops
Total Execution Time	22.35 s	25.70 s
System Throughput	2236.91 ops/s	1945.71 ops/sec
Average Request Latency	44.21 ms	51.02 ms

Tabella 5: workloadStressTest

Oss: Per **consistency level** = QUORUM, ALL potrebbero fallire le richieste per **timeout**, impostato di default a 2 secondi da Cassandra (comportamento osservabile nel *workloadConsistency*).

4.2 Test con Variazione del Numero di Nodi

In questa sezione saranno analizzati i risultati dei workload che prevedono la variazione del numero di nodi, in particolare abbiamo testato il comportamento nel range tra 1 e 4 nodi e *ReplicationFactor* = 3, per le ragioni esposte precedentemente (si veda sezione 4.1 per maggiori dettagli).

Di seguito le **performance** per ogni scenario:

Performance Metric	INTENSE WRITING	INTENSE READING
Total Completed Operations	2500 ops	2500 ops
Total Execution Time	1.09 seconds	1.43 seconds
System Throughput	2304.05 ops/sec	1750.81 ops/sec
Average Request Latency	10.46 ms	13.41 ms

Tabella 6: workloadScalability con 1 nodo

Performance Metric	INTENSE WRITING	INTENSE READING
Total Completed Operations	2500 ops	2500 ops
Total Execution Time	1.49 seconds	2.38 seconds
System Throughput	1683.11 ops/sec	1050.75 ops/sec
Average Request Latency	14.27 ms	22.75 ms

Tabella 7: workloadScalability con 2 nodi

Performance Metric	INTENSE WRITING	INTENSE READING
Total Completed Operations	2500 ops	2500 ops
Total Execution Time	1.83 seconds	5.01 seconds
System Throughput	1367.76 ops/sec	499.00 ops/sec
Average Request Latency	17.50 ms	48.24 ms

Tabella 8: workloadScalability con 4 nodi

Oss: In questo caso si può notare che la scalabilità é al contrario da quanto ci si potrebbe aspettare teoricamente, ovvero con una **proporzionalità linearmente crescente** (come spiegato in sezione 3.2).

Questo può essere attribuito al fatto che i container, pur simulando diversi nodi, risiedono comunque sulla stessa **infrastruttura hardware**, invalidando di fatto il risultato atteso. Uno sviluppo futuro del progetto sarebbe quindi quello di eseguire diversi nodi su diverse macchine virtuali per disaccoppiare di conseguenza anche l'hardware (come spiegato nelle conclusioni alla sezione 5).

Il risultati di `workloadMemStressTest` mostrano che le performance di scrittura non si degradano quasi per nulla, infatti il test mostra che è possibile inserire fino a 1 milione di utenti (circa 1GB di dati per nodo) senza che esse degradino.

L'ultimo workload da analizzare é `workloadFaultTolerance*` che si divide nelle rispettive componenti `workloadFaultToleranceONE.py`, `workloadFaultToleranceQUORUM.py`, `workloadFaultToleranceALL.py`.

In questo caso non ci saranno delle vere e proprie metriche di performance, quanto la capacità di poter portare a termine le richieste effettuate dall'utente, combinando il numero di nodi e i **consistency level**:

	ONE	QUORUM	ALL
1 Nodo	✓	fallisce	fallisce
2 Nodi	✓	fallisce	fallisce
3 Nodi	✓	✓	fallisce
4 Nodi	✓	✓	✓

Tabella 9: workloadFaultTolerance

4.3 Test su Macchine Distinte

Poiché l'esecuzione soprattutto di `workloadScalability` é molto influenzata dal fatto che i nodi si trovino sulla stessa macchina, seppur in container diversi, per questo workload sono stati eseguiti 3 nodi ($RF = 1$) ognuno su un **PC diverso** (tutto ciò é stato fatto

sul branch `lan` per differenziarsi dalla configurazione in locale).
I risultati di ciò sono i seguenti:

Performance Metric	INTENSE WRITING	INTENSE READING
Total Completed Operations	200000 ops	200000 ops
Total Execution Time	252.55 seconds	315.71 seconds
System Throughput	791.93 ops/sec	633.49 ops/sec
Average Request Latency	251.04 ms	313.86 ms

Tabella 10: workloadScalability con 1 nodo su 3 macchine distinte

Performance Metric	INTENSE WRITING	INTENSE READING
Total Completed Operations	200000 ops	200000 ops
Total Execution Time	261.27 seconds	423.88 seconds
System Throughput	765.50 ops/sec	471.83 ops/sec
Average Request Latency	258.53 ms	422.18 ms

Tabella 11: workloadScalability con 2 nodi su 3 macchine distinte

Performance Metric	INTENSE WRITING	INTENSE READING
Total Completed Operations	200000 ops	200000 ops
Total Execution Time	81.93 seconds	103.02 seconds
System Throughput	2441.00 ops/sec	1941.43 ops/sec
Average Request Latency	81.14 ms	102.05 ms

Tabella 12: workloadScalability con 3 nodi su 3 macchine distinte

5 Conclusioni

L'analisi sperimentale condotta su un cluster **Apache Cassandra** ha permesso di verificare sul campo le caratteristiche di scalabilità, tolleranza ai guasti e gestione dei carichi di lavoro in un contesto di social network distribuito. I test eseguiti hanno evidenziato come le prestazioni e la disponibilità del sistema siano strettamente correlate alla configurazione dei nodi e ai livelli di consistenza scelti.

5.1 Analisi delle Performance e Carico dei Nodi

Le prove effettuate sul cluster (configurato con 4 nodi e Replication Factor 3) hanno mostrato un sistema capace di gestire un throughput di circa **71.76 ops/s** con una latenza media contenuta di **6.84 ms** in condizioni di carico standard.

Un risultato di particolare rilievo riguarda l'efficienza dei nodi nella gestione delle letture: grazie all'implementazione dei **Bloom Filter**, la differenza di latenza tra una lettura andata a buon fine (*Read Hit*) e una su dati inesistenti (*Read Miss*) è risultata minima (rispettivamente **2.20 ms** e **2.31 ms**), dimostrando come i nodi possano evitare accessi inutili al disco. Tuttavia, la dimensione dei dati (*payload*) influisce drasticamente sulle prestazioni: il passaggio da un payload di 1 KB a uno di 100 KB ha causato un crollo

del throughput da **486.03 ops/s** a **90.46 ops/s**, con un aumento della latenza di oltre cinque volte.

5.2 Scalabilità e Vincoli Hardware

Contrariamente alle aspettative di una crescita lineare delle prestazioni, i test di scalabilità hanno mostrato che l'aggiunta di nodi in un ambiente containerizzato locale (su singola macchina con 16 GB di RAM) ha portato a un **degrado delle prestazioni**. Il throughput in scrittura è sceso da **2304.05 ops/s** con un singolo nodo a **1367.76 ops/s** con quattro nodi. Questo fenomeno evidenzia come l'overhead di gestione di multipli container Docker e la comunicazione inter-nodo saturino rapidamente le risorse hardware limitate (CPU i5 e memoria), rendendo controproducente l'espansione del cluster in assenza di un'infrastruttura distribuita reale.

Invece il test sui 3 nodi reali ha portato a delle conclusioni leggermente diverse: in condizioni di cluster integro (Tabella 12), il sistema mostra una scalabilità del throughput quasi perfettamente lineare, passando dalle **791.93 ops/s** del singolo nodo a **2441.00 ops/s** con tre nodi attivi (un incremento del $\sim 308\%$), con una latenza minima di **81.14 ms**. Avendo impostato $RF = 1$, l'assenza di traffico di replica permette infatti ai nodi di lavorare in parallelo senza overhead di sincronizzazione.

Al contrario, nei test con nodi offline (Tabelle 10 e 11) si nota un netto calo delle prestazioni, con il throughput che scende sotto le 800 ops/s e la latenza che si attesta intorno ai **250 ms**. Un aspetto interessante è che i risultati a 1 e 2 nodi sono quasi identici: questo andamento probabilmente è dovuto al fatto che il nodo spento per simulare il guasto era la macchina fisicamente più potente del cluster. Di conseguenza, nei test parziali il carico e la gestione degli errori di rete verso gli IP disconnessi sono gravati interamente sui due host rimasti attivi, che essendo meno performanti hanno appiattito le metriche complessive.

5.3 Tolleranza ai Guasti e Consistenza

Le prove di *Fault Tolerance* hanno confermato empiricamente il **CAP theorem** e la natura *Tunable Consistency* di Cassandra:

- **Consistency ONE**: Si è dimostrata la modalità più resiliente, permettendo al sistema di operare correttamente anche con un solo nodo attivo su quattro.
- **Consistency QUORUM**: Ha garantito l'operatività solo con la maggioranza dei nodi attivi (almeno 3 su 4), fallendo sistematicamente in scenari di guasto più severi.
- **Consistency ALL**: Ha mostrato la massima fragilità, dove il fallimento di un singolo nodo ha reso impossibile il completamento delle richieste.

In conclusione, l'esperienza ha dimostrato che Cassandra è estremamente efficace per applicazioni **Query-Driven** ad alta disponibilità, ma richiede una calibrazione accurata dei parametri di replica e un'infrastruttura hardware adeguata per poter beneficiare della scalabilità orizzontale. La gestione intelligente dei nodi e dei livelli di consistenza permette di bilanciare correttamente le esigenze di velocità e affidabilità richieste dal dominio applicativo dei social network.

6 How to Run

L'intero progetto è sviluppato in Python e richiede l'installazione di alcune dipendenze esterne, in particolare le librerie `cassandra-driver` e `Faker`. Per una corretta gestione delle dipendenze è consigliato creare un ambiente virtuale dedicato (`venv`), in modo da isolare le librerie del progetto dal resto del sistema.

La creazione dell'ambiente virtuale può essere effettuata tramite il comando:

```
python -m venv .venv
```

Successivamente, l'ambiente può essere attivato con:

Windows: `.venv\Scripts\activate`

Linux/macOS: `source .venv/bin/activate`

Una volta attivato l'ambiente virtuale, è possibile installare tutte le dipendenze richieste spostandosi nella cartella del progetto ed eseguendo:

```
pip install -r requirements.txt
```

In alternativa, le librerie possono essere installate singolarmente:

```
pip install cassandra-driver
```

```
pip install Faker
```

6.1 Avvio del cluster Cassandra tramite Docker

Per eseguire il cluster Cassandra è necessario aver installato Docker e Docker Compose. L'avvio dei container può essere effettuato tramite:

```
docker compose up -d
```

Una volta avviato il cluster, alcuni comandi utili per monitorarne lo stato sono:

Verifica stato nodi:

```
sudo docker exec -it cassandra-node1 nodetool status
```

Visualizzazione topologia cluster:

```
sudo docker exec -it cassandra-node1 nodetool describcluster
```

Nel caso in cui un nodo non venga avviato correttamente, è possibile riavviarlo mediante:

```
sudo docker restart cassandra-<nome_nodo>
```

Oss: Su sistemi Windows il prefisso `sudo` può essere omesso

6.2 Caricamento dello schema

Dopo l'avvio del cluster è necessario caricare lo schema Cassandra definito nel file `schema.cql`.

Windows:

```
Get-Content schema.cql | docker exec -i cassandra-node1 cqlsh
```

Linux/macOS:

```
sudo docker exec -i cassandra-node1 cqlsh < schema.cql
```

6.3 Popolamento del database

Una volta creato lo schema, il database può essere lanciato con dati sintetici eseguendo gli script Python relativi ai workload:

```
cd workloads
python3 <NOME_WORKLOAD>.py
```

6.4 Accesso alla shell CQL

Per interagire direttamente con Cassandra ed eseguire query manuali è possibile accedere alla shell CQL tramite:

```
sudo docker exec -it cassandra-node1 cqlsh
```

Ad esempio, una volta aperta la shell, è possibile interrogare la tabella degli utenti mediante:

```
SELECT username, email, first_name, last_name

FROM users;
```

6.5 Gestione dei container

Per arrestare temporaneamente i container mantenendo i volumi persistenti:

```
sudo docker compose stop
```

Per riavviare i container precedentemente arrestati:

```
sudo docker compose start
```

Per eliminare i container mantenendo i volumi:

```
sudo docker compose down
```

Per eliminare sia i container sia i volumi associati:

```
sudo docker compose down -v
```